

StrikeForce: A 2D Partially Observable Battle-Arena Testbed for Imitation Learning and Multi-Agent Research

Kasra Fouladi

Iran University of Science and Technology (IUST)

k4sr405@gmail.com

Github: <https://github.com/bistoyek21-ric/StrikeForce>

July 2026

Domain: Reinforcement/Imitation Learning Software, Game AI Testbeds, POMDPs

Keywords: testbed, partial observability, imitation learning, multiplayer client-server, demonstration collection, deterministic replay

Resource footprint: CPU-only (no GPU required); peak of 11% of a single Intel Core i7-8550U CPU core at the configured fixed simulation rate, with peak resident memory of 110 MB for the full client including rendering and all subsystems enabled.

1 Abstract

We present StrikeForce, an open-source, partially observable 2D battle-arena environment and client software implemented in C++17. StrikeForce couples a playable, account-backed game client (terminal and SFML-rendered graphical front ends) with a machine-readable agent-plugin interface, a fixed-size local observation encoding, a deterministic seeded random-number generator, frame-level replay logging, a client-server networking protocol with server-side match arbitration, and a lightweight HTTP-based checkpoint-transfer service backed by an SQLite state store for versioned, parent-child-tracked agent checkpoints. The environment supports five game modes (Solo, Timer, Squad, networked Battle Royal, and networked AI Battle Royal), an item/economy subsystem with 16 purchasable items across four categories, and a pluggable agent-injection seam that allows a researcher to substitute an arbitrary policy for the shipped example plugin without modifying the core simulation loop. The full client, with rendering and all subsystems enabled, consumes at most 11% of a single CPU core on a reference laptop and peaks at 110 MB of resident memory. This paper documents the environment’s design, software architecture, infrastructure, and resource footprint as a research testbed; it does not propose or evaluate a new learning algorithm, and the shipped example plugin is discussed only at the level of the interface it consumes.

2 Introduction

2.1 Motivation

Research on imitation learning and reinforcement learning under partial observability [7] benefits from environments that (i) are cheap to run in a headless/batch configuration, (ii) expose a fixed-size, well-documented observation and action interface, (iii) support exact or near-exact deterministic replay for debugging and dataset auditing, and (iv) make it easy to collect human demonstrations. Many existing testbeds satisfy a subset of these properties but were not designed around interleaved human-play data collection with a persistent account and ranking system. StrikeForce was built as a hobby/battle-royale-style game first and instrumented for machine learning second; this paper documents the resulting software as a testbed, independent of the specific learning method used on top of it.

2.2 Contribution

This paper’s contribution is the environment and its surrounding software infrastructure, not a learning algorithm. Concretely, we describe:

- the game/environment design (world, observation and action spaces, items, entities, game modes);
- the software architecture of the client, including its pluggable agent-injection seam;
- the infrastructure that supports research use (accounts, deterministic replay logging, a client–server networking protocol, a versioned checkpoint-transfer service, and an optional headless rendering configuration);
- the environment’s configured resource footprint, reported both as configuration constants declared in its source and as measured CPU and memory utilization on a reference machine;
- the agent-plugin interface and the shipped example plugin, described only at the level needed to document the interface contract.

2.3 Non-Goals

This paper makes no claim about learning performance, sample efficiency, or state-of-the-art results on any metric. Numeric values reported below are either configuration constants read directly from the shipped source code or direct measurements of the environment software itself, not benchmarked outcomes of any learning method.

3 Related Work

StrikeForce belongs to a broad family of game-based RL/IL testbeds. Brockman et al. [1] established the now-standard `step/reset` environment interface later extended to multi-agent settings by PettingZoo [11]. Pixel- and first-person-view partially observable arenas are exemplified by ViZDoom [8] and by StarCraft II’s learning environment [12]. Grid- or arena-style multi-agent battle games closer in spirit to StrikeForce include Pommerman [9] and Neural MMO [10], both of which emphasize many-agent, partially observable, procedurally-populated worlds. Progen [2] and Crafter [5] target procedural generalization and long-horizon survival respectively, and MineRL [4]

and the Unity ML-Agents toolkit [6] are notable for built-in support for human demonstration collection, a property StrikeForce also targets via its account-backed play client. Table 1 summarizes StrikeForce’s position along a small set of qualitative axes relative to a subset of these testbeds.

Environment	Partial Obs.	Multi-Agent	Network Play	Built-in Demo Collection	License
StrikeForce (this work)	Yes	Yes	Yes	Yes (accounts)	MIT
ViZDoom [8]	Yes	Limited	Engine-dependent	Via engine replay	MIT
StarCraft II LE [12]	Yes	Yes	Yes	Replay corpus	Custom (Blizzard/DeepMind)
Pommerman [9]	Yes	Yes	No	No	MIT
Neural MMO [10]	Yes	Yes	Limited	No	MIT
MineRL [4]	Yes	Limited	No	Yes (dataset)	MIT
Unity ML-Agents [6]	Configurable	Configurable	Configurable	Yes (demo recorder)	Apache-2.0

Table 1: Qualitative comparison of StrikeForce with a subset of related testbeds. Third-party rows describe general, publicly documented project properties and are not the subject of controlled benchmarking in this paper.

4 Environment Design

4.1 World

The world is a set of $F = 3$ stacked floors, each an $N \times M = 30 \times 100$ discrete grid of cells, connected by paired portal cells (portal-in/portal-out, tracked via a `portal_ind` index into a shared portal table). Each cell can hold at most one of: a player character, a zombie, a bullet in flight, a wall/block, a chest, or a portal endpoint; cell state is stored as an 11-bit flag vector (`std::bitset<11>`) per the `node` struct. Walls (#) and portals (~v/0) can be authored offline as plain-text ASCII floor maps (`./map/floor{1,2,3}.txt`), one character per cell, making custom map authoring a text-editing task rather than a code change.

Placeable obstacles are player-constructible at runtime: a player with block inventory can place a temporary wall (destructible after `lim_block = 1100` cumulative damage) or a portal endpoint (destructible after `lim_portal = 1000` cumulative damage), both tracked through the same damage-accumulation mechanism (`node::dmg`).

4.2 Entities and Capacity

The environment reserves fixed-size arrays for up to $H = 9000$ concurrent human characters, $Z = 9000$ zombies, and $B = 9000$ in-flight bullets/active portal slots, with a chest-spawn ceiling of $C = 9000$. These are compile-time capacity constants declared in `gameplay.hpp`, not measured runtime populations; typical matches use only a small fraction of this capacity.

Non-player entities are spawned probabilistically on a fixed schedule: chest-spawn attempts occur roughly every $pc = 10$ frames, zombie-NPC spawn attempts every $pz = 13$ frames, and human-NPC spawn attempts every $ph = 17$ frames (each gated additionally by a cell-occupancy check and, for zombies/humans, an available-slot check).

4.3 Observation Encoding

The environment exposes the full grid state at each frame; converting that state into a fixed-size observation tensor is the responsibility of the agent plugin. The shipped example plugin (`describe` in its `Custom.hpp`) produces an egocentric 31×31 crop of the current floor centered on the agent, encoded into $C = 32$ per-cell feature channels, giving a $32 \times 31 \times 31$ (30,752-element)

observation vector per frame. The 32 channels decompose as: 7 channels for object type (character, bullet, wall, chest, portal-in/out, temporary-damage flag), 8 channels for character situation (ally/enemy/NPC/zombie one-hot plus normalized kill count, block/portal inventory, and active-portal flag), 4 channels for passability/destructibility/normalized HP, 8 channels for attack-vector direction and normalized damage/effect/stamina of the occupying bullet or character, 3 channels for consumable-item stamina/effect/HP, and 2 channels for the occupant’s accrued damage/effect debuffs. Feature values above a threshold of 5.0 are passed through a bounded squashing transform, $10 - 10/(\beta(x - 5) + 2)$ with $\beta = 1$, before being emitted; values at or below the threshold are passed through unchanged. Cells that fall outside the map boundary are padded with a default empty node rather than wrapped toroidally.

The example plugin additionally reads one fixed cell (the agent’s own position, indices `FIXED_ROW=FIXED_COL=15`) directly as a non-spatial per-frame status vector, and receives the agent’s own previous executed action as a one-hot vector. These are consumption choices made by the example plugin, not properties the environment enforces on all agents.

4.4 Action Space

The full human command set recognized by the environment’s input handler (`obey`) spans 29 distinct single-character commands (`valid_commands` in `gameplay.hpp`): movement (4), turning (2), punch/shoot (2), four item-slot selectors per category (12), item use (1), block/portal placement (2), quit, backpack view, and command-list. The environment accepts commands at this character level; the conversion from an agent’s discrete action index to a character command is performed by the plugin’s `Custom.hpp` (`gameplay::bot`), not by the environment. The shipped example plugin operates over a 7-way discrete action space, `"+xeawsd"`, a strict subset of the full human command set; a research plugin is free to use a larger or different subset.

4.5 Items and Economy

Players accumulate in-game currency from kills and spend it in a shop on four categories of four items each (16 purchasable/upgradeable items plus a backpack-capacity upgrade), enumerated in Table 2. Each item exposes a name, volume, price, level, and category-specific stats, loaded from per-item text files under `./Items/` at startup.

Category	Item 1	Item 2	Item 3	Item 4
Consumable	energy_drink	first_aid_box	food_package	zombievaccine
Throwable	gas	flash_bang	acid_bomb	stinger
Cold weapon	push_dagger	wing_tactic	F_898	lochabreask
Warm weapon	AK_47	M416	MOSSBERG	AWM

Table 2: Shop item catalog (16 items across 4 categories), plus a separate backpack-capacity upgrade.

4.6 Item Statistics and Damage Model

Item catalog data is loaded from text files under `./Items/` at startup; each item carries a name, volume, price, level, and category-specific statistics. Consumables restore HP, stamina, or remove status effects; weapons carry damage, effect, range, and stamina cost; throwables additionally carry an area footprint. A consumable use (`Human::use`) restores HP/stamina, applies any effect to the

character’s accumulated debuff counter, decrements the item count, and updates the backpack’s occupied volume; if the count reaches zero the currently-selected slot is cleared.

Damage for melee and weapon attacks is resolved through a helper (`compute_damage(x, y)`) that returns a base damage value as a function of an attacker’s effective strength x and weapon range parameter y . Hit resolution (`Character::hit`) subtracts the bullet’s damage from the target’s HP and accumulates the bullet’s effect value into a separate debuff counter (`mindamage`), which is reset to the character’s baseline at the start of each match. When the selected action is a weapon or throwable whose stamina cost would push the character’s stamina below zero, the attack is refused and the character’s stamina is left unchanged; melee attacks do not check stamina.

Backpack capacity starts at 2000 volume units and is expanded through a purchasable upgrade (+200 volume per purchase); each character begins with a fixed block budget (8 blocks, 1 portal) that can be expanded through progression.

4.7 Game Modes

The client exposes five game modes from the main menu: **Solo** (survive and reach a kill quota, single player), **Timer** (survive and reach a kill quota within a time budget), **Squad** (5v5 against bot-controlled teams, local), **Battle Royal** (networked, human or agent-controlled), and **AI Battle Royal** (networked, agent-controlled). Solo and Timer expose 9 unlockable difficulty levels ($L = 9$) with per-level kill quotas of $5(i + 1)$ for level $i \in \{0, \dots, 8\}$ (Timer additionally imposes a $5(i + 1)$ -minute budget); Squad requires $10(i + 1)$ team kills and elimination of all rival-team players.

4.8 Progression and Ranking

Per-user match history and per-mode leaderboards (total, Solo, Timer, Squad) are persisted to disk and rendered through dedicated `history()` and `leaderboard()` views in the terminal UI.

4.9 Networking and Server-Side Match Arbitration

Networked modes use a TCP client–server protocol. On the client side, `gameplay::Client` authenticates with a server password, receives a shared timebase and serial number, exchanges per-player info blobs, and then exchanges one-byte command codes per opponent per frame. On the server side, `server.cpp` implements the match-arbitration loop:

- **Handshake.** The server listens on a user-specified TCP port, accepts one connection per player up to a configured player count n , and authenticates each connection against a plain-text session password before accepting it. Rejected connections receive an `R` byte; accepted connections receive an `A`.
- **Seeding.** A timebase and a 30-bit serial number are generated once at server startup and broadcast to every client; every client derives the same deterministic RNG stream from these values.
- **Team assignment.** The server operator specifies the number of teams m and assigns each player index to a team at startup.
- **Per-frame command relay.** Each frame, the server receives one command byte from every alive client and rebroadcasts it to every other alive (or newly eliminated) client. Elimination and disconnection are both detected through reserved command bytes (`~` for eliminated, `_` for disconnected).

- **Match resolution.** A match ends when the number of distinct teams still alive drops to one; the surviving team is declared the winner. If all players are eliminated, the match is declared a draw.
- **Optional logging.** When built with `-DLOGGING`, the server writes a per-match directory containing a match log, a header line with the timebase and serial number, and one per-player action stream file.

The server runs at a fixed receive timeout of 500 ms per player and uses parallel per-player I/O for the command-relay loop. Windows and POSIX socket paths branch on preprocessor guards.

5 Software Architecture

5.1 Module Layering

The client is organized as a chain of C++ headers, each including the previous layer, culminating in `main.cpp`: `basic.hpp` (platform I/O abstraction, colored console output, and the HTTP/7z backup client) → `random.hpp` (deterministic PRNG) → `Item.hpp` → `selected_agent.hpp` → `Agent.hpp/Modules.hpp` → `selected_custom.hpp` → `Custom.hpp` → `gameplay.hpp` (simulation loop, networking, rendering dispatch) → `Character.hpp` → `menu.hpp` → `enter.hpp` → `main.cpp`.

Game logic itself lives under the `Environment` namespace with sub-namespaces `Random`, `Item`, `Character`, and `Field`.

5.2 Extensibility Seam

Two indirection headers, `selected_agent.hpp` and `selected_custom.hpp`, each contain a single `#include` of a concrete implementation rather than logic of their own. This gives a research user a single point of substitution: pointing these two files at an alternative plugin class and an alternative `gameplay::bot/gameplay::prepare` pair swaps in a different policy or observation encoder without modifying `gameplay.hpp`, `menu.hpp`, `enter.hpp`, or `Character.hpp`. The environment is method-agnostic.

5.3 Agent Plugin Interface

An agent plugin is a class that exposes three methods used by the simulation loop:

- `predict(obs)` → action index, called once the plugin’s own observation encoder has produced the observation vector for the current frame;
- `update(action, imitate)`, called after a command is applied;
- `is_manual()`, which the simulation loop queries each frame to decide whether to read human keyboard input or plugin output.

The minimal contract is:

```
class Agent {
public:
    Agent(bool training = true);
    int  predict(const std::vector<float>& obs);
    void update(int action, bool imitate);
};
```

```

    bool is_manual();
};

```

The plugin is instantiated during `gameplay::prepare` and destroyed via `Human::deleteAgent` at match teardown. Episode serialization, when used, is performed by the plugin; the environment does not enforce any particular format on all plugins.

5.4 Rendering

Two front ends are compiled from the same simulation state: a terminal renderer using ANSI escape codes (`c_col` in `basic.hpp`), and an SFML-based `GraphicPrinter` that parses the same ANSI-colored text stream into a rendered window, running its draw loop on a dedicated thread synchronized via a mutex/condition-variable queue. A build-time `HIGHLY_OPTIMIZED` macro replaces `GraphicPrinter` with a no-op stub, disabling rendering entirely. In the non-headless path, the renderer paces itself to a fixed per-frame budget of 4×10^7 ns (40 ms), read from a `std::chrono::duration` on constant in `gameplay::print_game`.

The fixed pacing is a deliberate design choice for visual stability and to ensure the client remains usable on low-power machines; absolute throughput is therefore not a meaningful headline metric. The resource-footprint table in Section 7 accordingly reports CPU utilization at the configured rate and peak memory. On the reference laptop CPU used for measurement (Intel Core i7-8550U @ 1.80 GHz), the full client at its configured rate consumes at most 11% of a single core.

5.5 Build System and Dependencies

The client targets C++17 and is built with the compile invocations documented in-source, e.g. `g++ -std=c++17 main.cpp -o app -ltorch -ltorch_cpu -ltorch_cuda -lc10 -lc10_cuda -lsfml -graphics -lsfml-window -lsfml-system` (with `-lws2_32` added on Windows). Dependencies are: a C++17 toolchain, LibTorch (CPU and/or CUDA builds), SFML, and, for the backup subsystem, external `curl` and `7z` binaries. The server (`server.cpp`) depends on a C++17 toolchain and a POSIX or Winsock TCP stack only.

The intended workflow is compile-and-run: the environment is distributed as source, and the target artifact is a directly executable binary, not a packaged or installable build. The agent plugin is compiled together with the client in a single translation unit; a researcher iterating on the plugin simply recompiles `main.cpp` and re-runs the resulting executable. This design choice keeps the workflow light (no build system, no dependency graph, no installation step) and matches the iterative nature of agent development, where the plugin is expected to change frequently while the environment remains fixed.

Build-time feature flags are centralized in `macros.hpp` (e.g. `CROWDSOURCED_TRAINING`, `SLOWMOTION`, `HIGHLY_OPTIMIZED`, `DATASET` path, `DATA_GATHERING_MODE`, `AUTOMATIC`), letting a single codebase be compiled into a playable client, a data-collection client, a headless simulation client, or a networked match server.

6 Infrastructure

6.1 Accounts

Sign-up/sign-in is handled by `enter.hpp` with passwords hashed via an in-repository SHA-256 implementation before being written to `./accounts/pass.txt`; usernames are stored in a parallel `./accounts/users.txt`. A “remember me” option persists a username and timestamp to `./accounts/saved_login.txt`; a saved login is honored for up to $7 \times 60 \times 24 \times 60 = 604,800$

seconds (7 days). Each account gets a dedicated game directory holding a per-user info/character file and a per-user match-history log.

6.2 Deterministic Simulation and Replay

Randomness is drawn from a custom generator (`Environment::Random`, `random.hpp`) seeded via `_srand(tb, serial_number)` from a timebase and a serial number; the same seed pair reproduces the same pseudo-random sequence, and in networked play the timebase/serial number are transmitted from server to client so that all participants share one RNG stream. Matches can additionally be logged frame-by-frame to `.sf_sample` files, and `replay_mode` re-consumes an `.sf_sample` file instead of live input, re-seeding the RNG from the values stored at the head of the file.

6.3 Crowdsourced Demonstration Collection

A `CROWDSOURCED_TRAINING` build flag changes input-handling behavior in `gameplay::my_command` to standardize play sessions intended for demonstration collection. Collected episodes are written under a configurable `DATASET` path, with subdirectories keyed by mode and online flag and filenames keyed by wall-clock timestamp. The plugin is responsible for the episode serialization format; the environment only schedules and paces the collection.

6.4 Checkpoint-Transfer Service

The client-side backup subsystem in `basic.hpp` implements two shelled-out HTTP operations against a fixed server URL: `request_and_extract_backup(dir, bot_code)`, which downloads `backup.zip` from `/StrikeForce/api/request_backup?bot=<bot_code>` and extracts it with `7z`; and `zip_and_return_backup(dir)`, which zips a directory and POSTs it to `/StrikeForce/api/return_backup`. The server side is implemented as a Flask application (`server.py`) with an SQLite state store (`database.py`), organized around four tables:

- `crypto_keys` — a rolling history of AES-256-CBC key/IV pairs used to encrypt backup metadata;
- `bots` — the set of registered plugin names, each of which owns a directory of backups;
- `backups` — one row per registered checkpoint, keyed by a random 256-bit serial and carrying a `parent_serial` field that records lineage;
- `branches` — outstanding `(serial, epoch, password_hash)` triples issued to clients when a backup is handed out, consumed when the client returns a trained checkpoint.

The service exposes two endpoint groups:

Admin endpoints. Authenticated by an admin key whose SHA-256 hash is compared against a stored constant. Operations include `add_bot` (validated against `^[a-zA-Z0-9_-]{3,50}$`), `delete_bot`, `add_backup` (accepts an uploaded zip, extracts it after rejecting any archive entry with an absolute path or `..` component, generates a fresh serial and password, and writes an AES-256-CBC-encrypted `metadata.enc`), `delete_backup`, `get_crypto`, and `set_crypto` (with hex-length validation for the 32-byte key and 16-byte IV).

Client endpoints. `request_backup` selects a random backup from the requested bot's pool, mints an `(epoch, password)` pair, records its SHA-256 hash as a new branch in the database,

updates the backup’s encrypted metadata with the new epoch and password, and streams the zipped backup directory to the client. `return_backup` accepts an uploaded zip, decrypts the embedded metadata, validates the `(serial, epoch, password_hash)` triple against the outstanding-branch table, consumes that branch, and then either replaces the existing backup (if the metadata is unchanged) or creates a new backup whose `parent_serial` is the returned backup’s serial. This is the mechanism by which a lineage of checkpoints is maintained: each returned training round spawns a new node in the backup tree with a pointer to its parent, so the full ancestry of any given checkpoint is reconstructible from the database.

A global upload-size limit of 100 MB is enforced on the server, and all endpoint handlers are wrapped in a shared exception handler that logs to `app.log` and returns a structured JSON error response.

7 Resource Footprint

Table 3 reports two categories of quantities: configuration constants declared directly in the reviewed source files, and direct measurements of the environment software on a reference machine. The measurements were taken with the full client running at its configured fixed pacing and rendering enabled; peak CPU utilization was 11% of a single core on an Intel Core i7-8550U @ 1.80 GHz, and peak resident memory was 110 MB. No agent plugin was loaded during measurement.

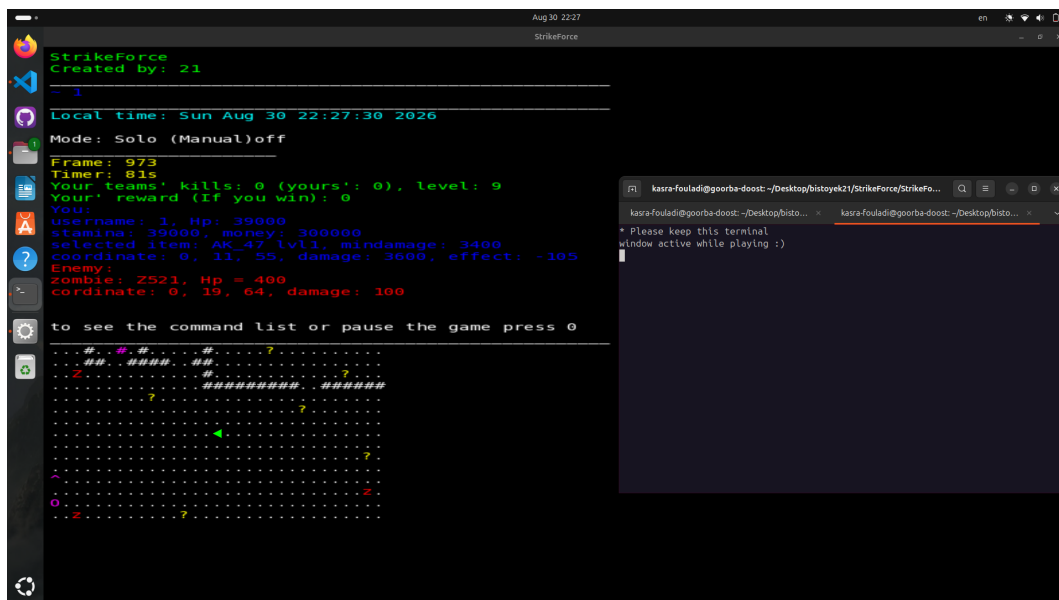


Figure 1: StrikeForce client, graphical front end.

8 Example Agent Plugin

The repository ships one example agent plugin under `bots/` to demonstrate the three-method plugin interface of Section 5.3. It is loaded through the `selected*.hpp` seam described in Section 5 and consists of a policy network, a runtime wrapper that loads a trained checkpoint and exposes the interface, and a rule-based inference-time behavior mask over the action logits. The algorithmic

Quantity	Value (source)
Floors	$F = 3$
Floor grid	$N \times M = 30 \times 100$ cells
Local observation window (example plugin)	31×31 cells, egocentric crop
Observation channels (example plugin)	$C = 32$
Observation vector size (example plugin)	$32 \times 31 \times 31 = 30,752$ floats/frame
Example plugin action space	7 discrete actions
Full human command set	29 distinct single-key commands
Max concurrent humans	$H = 9000$ (array capacity, not typical occupancy)
Max concurrent zombies	$Z = 9000$ (array capacity)
Max concurrent bullets/portal slots	$B = 9000$ (array capacity)
Chest-spawn ceiling	$C_{\text{chest}} = 9000$
Chest-spawn attempt interval	every $pc = 10$ frames
Zombie-spawn attempt interval	every $pz = 13$ frames
Human-NPC spawn attempt interval	every $ph = 17$ frames
Initial backpack capacity	2000 volume units
Backpack upgrade per purchase	+200 volume units
Default blocks per match	8
Default portals per match	1
Block destructibility threshold	1100 cumulative damage
Portal destructibility threshold	1000 cumulative damage
Nominal render frame budget	40 ms, non-headless build only
Headless build	HIGHLY_OPTIMIZED disables rendering entirely
Server receive timeout	500 ms per player
Saved-login validity window	604,800 s (7 days)
API server upload limit	100 MB per request
GPU requirement	None (environment is CPU-only)
Peak CPU utilization	11% of a single core (Intel Core i7-8550U @ 1.80 GHz)
Peak resident memory	110 MB

Table 3: Configuration constants and declared capacities read directly from source, together with measured CPU and memory utilization of the environment software on a reference machine. Plugin inference cost is excluded.

content of the example plugin is out of scope for this paper; it is provided only as a reference for how the interface is meant to be used.

9 Intended Use Cases

The design choices documented above suggest StrikeForce is best suited to: (i) imitation-learning research in partially observable, pseudo-Markovian settings; (ii) research on post-hoc behavioral correction or consolidation of a trained policy, given the checkpoint-transfer service and the plugin-substitution seam; (iii) research requiring paired human demonstrations and agent rollouts from the same environment, given the account system, crowdsourced-training build flag, and plugin-controlled episode-serialization format; and (iv) debugging and auditing of collected trajectories via deterministic seeded replay. These are use cases the shipped software supports, not claims about how well any particular method performs on them.

10 Limitations

- **Resource measurements are from a single reference CPU.** The reported 11% single-core utilization and 110 MB peak resident memory were measured on an Intel Core i7-8550U @ 1.80 GHz; absolute numbers on other hardware will differ, and we do not report cross-platform profiling here.
- **Backup metadata is encrypted, not authenticated.** The checkpoint-transfer service encrypts metadata with AES-256-CBC using a server-held key. This is a confidentiality mechanism, not an integrity mechanism; a client that holds the encrypted metadata blob cannot forge a valid one without the key, but the scheme does not provide a cryptographic MAC over the ciphertext. Operational security therefore relies on the server holding the key and on TLS termination at the deployment layer.

11 Licensing and Availability

Every source file reviewed for this paper carries an MIT License header (Copyright © bistoyek21 R.I.C., 2024–2026). The environment is available as a GitHub repository [3].

12 Conclusion

We described StrikeForce as a software artifact: a partially observable, item-and-economy-bearing 2D battle-arena client with a fixed-size observation/action interface, a pluggable agent-plugin seam, deterministic seeded simulation with frame-level replay logging, an account and leaderboard system, a client–server networking protocol with server-side match arbitration, and a versioned checkpoint-transfer service with parent–child lineage. The full client at its configured pacing consumes at most 11% of a single CPU core and peaks at 110 MB of resident memory on a reference laptop. We deliberately excluded learning-algorithm content; the shipped example plugin is documented only at the level of the interface it consumes. We hope documenting the environment on its own terms makes it easier for other researchers to reuse, extend, or critically evaluate as a testbed independent of any specific method built on top of it.

References

- [1] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. arXiv:1606.01540, 2016. URL <https://arxiv.org/abs/1606.01540>. arXiv:1606.01540.
- [2] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 2048–2056, 2020. URL <https://arxiv.org/abs/1912.01588>. arXiv:1912.01588.
- [3] Kasra Fouladi and bistoyek21 R.I.C. Strikeforce: Ai agent battle arena. GitHub repository, 2026. URL <https://github.com/bistoyek21-ric/StrikeForce>. MIT License.
- [4] William H. Guss, Brandon Houghton, Nicholay Topin, Phillip Wang, Cayden Codell, Manuela Veloso, and Ruslan Salakhutdinov. Minerl: A large-scale dataset of minecraft demonstrations.

- In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2442–2448, 2019. doi: 10.24963/ijcai.2019/338.
- [5] Danijar Hafner. Benchmarking the spectrum of agent capabilities. arXiv:2109.06780, 2021. URL <https://arxiv.org/abs/2109.06780>. arXiv:2109.06780.
 - [6] Arthur Juliani, Vincent-Pierre Berges, Eloi Vekay, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. Unity: A general platform for intelligent agents. arXiv:1809.02627, 2018. URL <https://arxiv.org/abs/1809.02627>. arXiv:1809.02627.
 - [7] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X.
 - [8] Michał Kempka, Marek Wydmuch, Grzegorz Runc, Jakub Toczek, and Wojciech Jaśkowski. Vizdoom: A doom-based ai research platform for visual reinforcement learning. In *2016 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–8, 2016. doi: 10.1109/CIG.2016.7860433.
 - [9] Cinjon Resnick, Wes Eldridge, David Ha, Denny Britz, Jakob Foerster, Julian Togelius, Kyunghyun Cho, and Joan Bruna. Pommerman: A multi-agent playground. arXiv:1809.07124, 2018. URL <https://arxiv.org/abs/1809.07124>. arXiv:1809.07124.
 - [10] Joseph Suarez, Yilun Du, Phillip Isola, and Igor Mordatch. Neural mmo: A massively multiagent game environment for training and evaluating intelligent agents. arXiv:1903.00784, 2019. URL <https://arxiv.org/abs/1903.00784>. arXiv:1903.00784.
 - [11] J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Christian Horsch, Rodrigo Perez-Vicente, Niall Williams, Yashas Lokesh, and Praveen Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL <https://arxiv.org/abs/2009.14471>. arXiv:2009.14471.
 - [12] Oriol Vinyals, Timo Ewalds, Sergey Bartunov, Petko Georgiev, Alexander Sasha Vezhnevets, Michelle Yeo, Alireza Makhzani, Heinrich Küttler, John Agapiou, Julian Schrittwieser, et al. Starcraft ii: A new challenge for reinforcement learning. arXiv:1708.04782, 2017. URL <https://arxiv.org/abs/1708.04782>. arXiv:1708.04782.